

SchrotbergerDSC670Milestone4

August 9, 2026

```
[1]: import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

[2]: def generate_recipe(ingredients, allergens):
    prompt = f"""
You are a dietician and chef. Using ONLY the provided ingredients create a
    ↪recipe that includes a full ingredient list and instructions on how to make
    ↪it.

Allowed ingredients: {"", ".join(ingredients)}

Avoid ALL of these allergens or restricted ingredients:
{"", ".join(allergens)}

Rules:
- You can add more ingredients than those provided to complete the recipe.
- Do NOT include any ingredient that is an allergen or contains an allergen.
- If a recipe includes an allergen, suggest a safe alternative to substitute
    ↪the ingredient with.
- Output must be in JSON with the following structure:

{{
  "title": "",
  "description": "",
  "ingredients": ["item 1", "item 2"],
  "steps": ["step 1", "step 2", "step 3"],
  "allergy_safe_for": ["list of allergens avoided"]
}}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
```

```

        temperature=0.4
    )

    return response.choices[0].message.content

```

```

[3]: def main():
    print("\nCooking with Caitie\n")
    print("\nTo create a recipe please enter the following:\n")

    ingredients = input("Available ingredients (comma-separated): ")
    allergens = input("Allergens or ingredients to avoid (comma-separated): ")

    ingredients_list = [i.strip() for i in ingredients.split(",") if i.strip()]
    allergens_list = [a.strip() for a in allergens.split(",") if a.strip()]

    print("\nGenerating recipe...\n")
    recipe = generate_recipe(ingredients_list, allergens_list)

    print(recipe)

if __name__ == "__main__":
    main()

```

Cooking with Caitie

To create a recipe please enter the following:

Available ingredients (comma-separated): chicken, rice
 Allergens or ingredients to avoid (comma-separated): peanuts

Generating recipe...

```

```json
{
 "title": "Lemon Herb Chicken with Rice",
 "description": "A flavorful and healthy dish featuring tender chicken
 marinated in lemon and herbs, served over fluffy rice.",
 "ingredients": [
 "2 chicken breasts",
 "1 cup rice",
 "2 cups chicken broth",
 "2 tablespoons olive oil",
 "1 lemon (juiced and zested)",
 "2 cloves garlic (minced)",

```

```

 "1 teaspoon dried oregano",
 "1 teaspoon dried thyme",
 "Salt and pepper to taste",
 "Fresh parsley (for garnish)"
],
"steps": [
 "In a bowl, combine olive oil, lemon juice, lemon zest, minced garlic,
oregano, thyme, salt, and pepper to create a marinade.",
 "Add the chicken breasts to the marinade, ensuring they are well-coated. Let
them marinate for at least 30 minutes.",
 "In a pot, bring the chicken broth to a boil. Add the rice, reduce heat to
low, cover, and simmer for about 15-20 minutes or until the rice is cooked and
fluffy.",
 "While the rice is cooking, heat a skillet over medium heat. Remove the
chicken from the marinade and cook in the skillet for about 6-7 minutes on each
side, or until fully cooked and golden brown.",
 "Once the chicken is cooked, remove it from the skillet and let it rest for
a few minutes before slicing.",
 "Serve the sliced chicken over the cooked rice and garnish with fresh
parsley."
],
"allergy_safe_for": ["peanuts"]
}
...

```

```

[4]: import pandas as pd

cookbook = pd.read_csv('//wsl$/Ubuntu/home/caiti/projects/cookbook/recipes/
↳RAW_recipes.csv')

```

```

[4]:

```

	name	id	minutes	\
0	arriba baked winter squash mexican style	137739	55	
1	a bit different breakfast pizza	31490	30	
2	all in the kitchen chili	112140	130	
3	alouette potatoes	59389	45	
4	amish tomato ketchup for canning	44061	190	

  

	contributor_id	submitted	\
0	47892	2005-09-16	
1	26278	2002-06-17	
2	196586	2005-02-25	
3	68585	2003-04-14	
4	41706	2002-10-25	

  

	tags	\
0	['60-minutes-or-less', 'time-to-make', 'course...	
1	['30-minutes-or-less', 'time-to-make', 'course...	

```

2 ['time-to-make', 'course', 'preparation', 'mai...
3 ['60-minutes-or-less', 'time-to-make', 'course...
4 ['weeknight', 'time-to-make', 'course', 'main-...

 nutrition n_steps \
0 [51.5, 0.0, 13.0, 0.0, 2.0, 0.0, 4.0] 11
1 [173.4, 18.0, 0.0, 17.0, 22.0, 35.0, 1.0] 9
2 [269.8, 22.0, 32.0, 48.0, 39.0, 27.0, 5.0] 6
3 [368.1, 17.0, 10.0, 2.0, 14.0, 8.0, 20.0] 11
4 [352.9, 1.0, 337.0, 23.0, 3.0, 0.0, 28.0] 5

 steps \
0 ['make a choice and proceed with recipe', 'dep...
1 ['preheat oven to 425 degrees f', 'press dough...
2 ['brown ground beef in large pot', 'add choppe...
3 ['place potatoes in a large pot of lightly sal...
4 ['mix all ingredients& boil for 2 1 / 2 hours ...

 description \
0 autumn is my favorite time of year to cook! th...
1 this recipe calls for the crust to be prebaked...
2 this modified version of 'mom's' chili was a h...
3 this is a super easy, great tasting, make ahea...
4 my dh's amish mother raised him on this recipe...

 ingredients n_ingredients
0 ['winter squash', 'mexican seasoning', 'mixed ... 7
1 ['prepared pizza crust', 'sausage patty', 'egg... 6
2 ['ground beef', 'yellow onions', 'diced tomato... 13
3 ['spreadable cheese with garlic and herbs', 'n... 11
4 ['tomato juice', 'apple cider vinegar', 'sugar... 8

```

```

[6]: ALLERGEN_MAP = {
 "dairy": ["milk", "cheese", "cream", "butter", "yogurt", "ghee"],
 "nuts": ["peanut", "almond", "walnut", "pecan", "cashew", "hazelnut", "nut"],
 "gluten": ["wheat", "flour", "bread", "pasta", "noodle"],
 "soy": ["soy", "tofu", "soybean", "tamari"],
 "shellfish": ["shrimp", "crab", "lobster", "shellfish", "clam"],
 "egg": ["egg"],
}

```

```

[7]: def detect_allergens(ingredients):
 found = set()
 for allergen, keywords in ALLERGEN_MAP.items():
 for kw in keywords:
 if any(kw.lower() in ing.lower() for ing in ingredients):
 found.add(allergen)

```

```
return list(found)
```

```
[8]: import json
import random

def build_allergen_finetune_dataset(cookbook, n=5000):
 dataset = []

 for _, row in cookbook.head(n).iterrows():
 ingredients = row["ingredients"]
 steps = row["steps"]
 title = row["name"]
 tags = row["tags"]

 # Detect allergens present in the recipe
 recipe_allergens = detect_allergens(ingredients)

 # Randomly choose allergens to avoid (teaches avoidance behavior)
 avoid = random.sample(list(ALLERGEN_MAP.keys()), k=random.randint(1, 3))

 # Build user prompt
 user_prompt = (
 f"Ingredients available: {'', '.join(ingredients)}. "
 f"Allergens to avoid: {'', '.join(avoid)}. "
 f"Create a safe recipe in JSON format."
)

 # Build assistant output
 assistant_output = {
 "title": title,
 "ingredients": ingredients,
 "instructions": steps,
 "tags": tags,
 "allergens_detected": recipe_allergens,
 "safe_for": [a for a in avoid if a not in recipe_allergens],
 "substitutions": {
 allergen: f"Replace {ALLERGEN_MAP[allergen][0]} with a safe_
↪alternative"
 for allergen in avoid
 }
 }

 dataset.append({
 "messages": [
 {"role": "user", "content": user_prompt},
 {"role": "assistant", "content": json.dumps(assistant_output)}
]
 })
```

```

 })

 return dataset

dataset = build_allergen_finetune_dataset(cookbook)

```

[8]: 5000

```

[9]: with open("kaggle_allergen_finetune.jsonl", "w") as f:
 for item in dataset:
 f.write(json.dumps(item) + "\n")

```

Saved kaggle\_allergen\_finetune.jsonl

```

[10]: file = client.files.create(
 file=open("kaggle_allergen_finetune.jsonl", "rb"),
 purpose="fine-tune"
)

```

Response:

```

FileObject(id='file-AtEvFTvpboaRoHVAXVqizC', bytes=9201255,
created_at=1778458613, filename='kaggle_allergen_finetune.jsonl', object='file',
purpose='fine-tune', status='processed', expires_at=None, status_details=None)
Training file ID: file-AtEvFTvpboaRoHVAXVqizC
Training file name: kaggle_allergen_finetune.jsonl

```

```

[12]: ft = client.fine_tuning.jobs.create(
 training_file=file.id,
 model="gpt-3.5-turbo",
 suffix="allergen-recipe-model"
)

```

Finetuning job ID: ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP

```

[13]: ft_jobs = client.fine_tuning.jobs.list()

 for ft_job in ft_jobs:
 print(ft_job.id, ft_job.status)

```

```

ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP validating_files
ftjob-d61aN6vD0jQoMRp7zTD3g2Nv succeeded
ftjob-wEjiRF3Hhxxh9Zry5DJ4r6aI succeeded
ftjob-sXeNpNpZl4v738obZMtTMnj2 succeeded
ftjob-Qmf5HcpvATVCyaijP2GYkbbq succeeded
ftjob-9hfKV7gLOGajJrfFptTWVGJ7 succeeded
ftjob-lwXy3ykDi63uTRUV0zjA4hGL failed

```

```

[14]: ft_job_events = client.fine_tuning.jobs.list_events(
 fine_tuning_job_id="ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP",

```

```

 limit=2
)

 for ft_job_event in ft_job_events:
 print(ft_job_event.id, ft_job_event.message)

```

ftevent-FUYlY2a4vJHBI4ibT5Usa7IQ Validating training file: file-  
AtEvFTvpboaRoHVAXVqizC  
ftevent-mcLHvXjLVxczRllqGGNwCFw Created fine-tuning job: ftjob-  
wmQ8U7GN1eFvtjvLtRF7ftqP

```

[16]: # Track fine-tuning job status and wait until complete
import time
job = "ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP"

from IPython.display import clear_output

start_time = time.time()

Get the status of our fine-tuning job.
ft_job = client.fine_tuning.jobs.retrieve(job)
status = ft_job.status

If the job isn't done yet, poll it every 10 seconds.
while status not in ["succeeded", "failed"]:
 time.sleep(10)

 ft_job = client.fine_tuning.jobs.retrieve(job)
 print(ft_job)
 status = ft_job.status
 print("Elapsed time: {} minutes {} seconds".format(int((time.time() -
↪start_time) // 60), int((time.time() - start_time) % 60)))
 print(f'Status: {status}')

 clear_output(wait=True)

print(f'Fine-tuning job {job} finished with status: {status}')

```

Fine-tuning job ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP finished with status: succeeded

```

[17]: # List all fine-tuning jobs for this resource.
print('Checking other fine-tune jobs for this resource.')
List all the FT jobs
ft_jobs = client.fine_tuning.jobs.list()

for ft_job in ft_jobs:
 print(ft_job.id, ft_job.status)

```

```

job = client.fine_tuning.jobs.retrieve("ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP")
fine_tuned_model = job.fine_tuned_model
print(job.fine_tuned_model)

```

Checking other fine-tune jobs for this resource.

```

ftjob-wmQ8U7GN1eFvtjvLtRF7ftqP succeeded
ftjob-d61aN6vD0jQoMRp7zTD3g2Nv succeeded
ftjob-wEjiRF3Hhhxh9Zry5DJ4r6aI succeeded
ftjob-sXeNpNpZl4v738obZMtTMnj2 succeeded
ftjob-Qmf5HcpvATVCyaijP2GYkbbq succeeded
ftjob-9hfKV7gL0GajJrfFptTWVGJ7 succeeded
ftjob-lwXy3ykDi63uTRUV0zjA4hGL failed
ft:gpt-3.5-turbo-0125:personal:allergen-recipe-model:De9dV0V1

```

```

[25]: def generate_recipe2(ingredients2, allergens2):
 prompt2 = f"""
 You are a dietician and chef. Using ONLY the provided ingredients create a
 ↪recipe that includes a full ingredient list and instructions on how to make
 ↪it.

 Allowed ingredients: {", ".join(ingredients2)}

 Avoid ALL of these allergens or restricted ingredients:
 {", ".join(allergens2)}

 Rules:
 - You can add more ingredients than those provided to complete the recipe.
 - Do NOT include any ingredient that is an allergen or contains an allergen.
 - If a recipe includes an allergen, suggest a safe alternative to substitue
 ↪the ingredient with.
 - Output must be in JSON with the following structure:

 {{
 "title": "",
 "description": "",
 "ingredients": ["item 1", "item 2"],
 "steps": ["step 1", "step 2", "step 3"],
 "allergy_safe_for": ["list of allergens avoided"]
 }}

 """

 response2 = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[{"role": "user", "content": prompt2}],
 temperature=0.4
)

```



```
return response2.choices[0].message.content
```

```
[27]: def main2():
 print("\nCooking with Caitie\n")
 print("\nTo create a recipe please enter the following:\n")

 ingredients2 = input("Available ingredients (comma-separated): ")
 allergens2 = input("Allergens or ingredients to avoid (comma-separated): ")

 ingredients_list2 = [i.strip() for i in ingredients2.split(",") if i.
↪strip()]
 allergens_list2 = [a.strip() for a in allergens2.split(",") if a.strip()]

 print("\nGenerating recipe...\n")
 recipe2 = generate_recipe2(ingredients_list2, allergens_list2)

 print(recipe2)

if __name__ == "__main__":
 main()
```

Cooking with Caitie

To create a recipe please enter the following:

Available ingredients (comma-separated): chicken, rice

Allergens or ingredients to avoid (comma-separated): peanuts

Generating recipe...

```
```json
{
  "title": "Chicken and Rice Pilaf",
  "description": "A flavorful and comforting dish made with tender chicken and
fluffy rice, seasoned to perfection.",
  "ingredients": [
    "2 cups of rice",
    "1 pound of chicken (boneless, skinless, cut into bite-sized pieces)",
    "4 cups of chicken broth",
    "1 medium onion (finely chopped)",
    "2 cloves of garlic (minced)",
    "2 tablespoons of olive oil",
    "1 teaspoon of salt",
    "1/2 teaspoon of black pepper",
```

```

    "1 teaspoon of paprika",
    "1/2 teaspoon of dried thyme",
    "1/2 teaspoon of dried parsley"
  ],
  "steps": [
    "In a large skillet, heat the olive oil over medium heat. Add the chopped onion and minced garlic, sautéing until the onion is translucent.",
    "Add the chicken pieces to the skillet, season with salt, pepper, paprika, thyme, and parsley. Cook until the chicken is browned on all sides.",
    "Stir in the rice, making sure to coat it with the oil and seasonings. Pour in the chicken broth and bring to a boil. Reduce the heat to low, cover, and let it simmer for about 18-20 minutes or until the rice is cooked and has absorbed the liquid."
  ],
  "allergy_safe_for": ["peanuts"]
}

```